Monte Carlo Simulation - Portfolio loss and importance sampling

Thomas Sejer Canham

October 23, 2026

Aarhus University

Contents

1 Introduction **1**

2 Crude Monte Carlo Simulation (a) **1**

3 Control Variable (b) **2**

4 Importance-Sampling (c) **5**

5 Comparison of the methods (d) **7**

6 Closed form expressison of $\mathbb{P}(L > x)$ for the given parameters (e) **9**
6.1 Is this true for other distributions? . . . 10
6.2 Generalization of crude and control- MCS to other distributions . . . 10

7 Cholesky Decomposition (f) **10**
7.1 Eigen decomposition of $\tilde{C}^T\tilde{C}$ . . . 11
7.2 New expression of L . . . 11

8 Repetition of (a) and (b) with new parameters **12**
8.1 Repetition of (a) . . . 12
8.2 Repetition of (b) . . . 12

9 Exponential tilting (g) **12**

10 Cumlant generating function of L (h) **14**

11 Choosing θ based on the cumulant generating function (i) **15**
11.1 Distribution under the shifted measure . . . 15
11.2 Algorithm . . . 16

12 Stratified sampling (j) **17**

13 Appendix **18**

Declaration of use of GAI tools

I used generative artificial intelligence (GAI) I have used GPT 5.6 Deepthink

---

I used GAI tools in the following way

- Code debugging.
- Generation of code for creating plots and figures.
- Autocompletion of code snippets and text.
- Guidance and help with understanding specfic concepts and methods.

Example prompts could be:

- "What causes the numpy broadcasting error in this code snippet?: (Paste of code snippet)"
- "Generate plotting code the visualize the confidence intervals of the 30 simulation runs i compute in this code snippet: (Paste of code snippet)"
- "Help me understand how to complete the square in a matrix expression."

1 Introduction

In this project, we will investigate the use of Monte Carlo simulation (MCS) to estimate the exceedance probability $\mathbb{P}(L > x)$ of a loss

$$L = \delta^T \Delta S + \Delta S^T \Gamma \Delta S,$$

where, $delta \in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ and $\Delta S \sim \mathcal{N}(0, \Sigma_S)$ is a (multivariate) normal random variable.

In the parts (a), (b) and (c), we will focus on the one-dimensional case, $m = 1$. These three parts will compare crude MCS with MCS where we introduce control variates and importance sampling respectively. In part (d), the differences of these methods are visuallized by comparison of the confidence intervals of the estimates that are obtained from each method. In part (e) we derive a closed form expression for the exceedance probability $\mathbb{P}(L > x)$ for the given parameters, and discuss how this can be generalized to other distributions.

In parts (f), (g) and (h), we will consider the two-dimensional case, $m = 2$. In part (f), we will perform a Cholesky decomposition of the covariance matrix Σ_S, and use this to rewrite L as a function of a standard normal random variable. In part (g), we will implement importance sampling using exponential tilting, in part (h) we derive results for the cumulant generating function of L, and in part (i) we will discuss how to choose the tilting parameter θ based on the cumulant generating function of L.

Finally, in part (j), we dicuss how stratified sampling conceptually could be used to estimate the exceedance probability.
Parameters for the various codes snippets can be seen in the appendix in Listing 8.

2 Crude Monte Carlo Simulation (a)

In this section, we will implement a crude Monte Carlo simulation to estimate the exceedance probability $\mathbb{P}(L > x)$ for the one-dimensional case ($m = 1$). We are interested in $P(L > x)$. We may introduce the indicator function $\mathbb{I}(L > x)$ of the event $L > x$. If we compute the expectation of the indicator function, we obtain the exceedance probability

$$\mathbb{P}(L > x) = \mathbb{E}[\mathbb{I}(L > x)] = 0 \cdot \mathbb{P}(L > x) + 1 \cdot \mathbb{P}(L > x) = \mathbb{P}(L > x),$$

by the bernoulli property of the indicator function. We estimate the exceedance probability by use of the following Monte Carlo estimator

$$\hat{\ell} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(L_i > x) \xrightarrow[N \to \infty]{a.s} \mathbb{E}[\mathbb{I}(L > x)],$$

by the law of large numbers. The crude Monte Carlo simulation is implemented in the code snippet seen in Listing 1. A estimate of

$$\hat{\ell} \approx 0.0103$$

is found with a variance of $s^2 = 1.03 \cdot 10^{-7}$, for $N = 10^5$.

Listing 1: Python code for crude MCS

```
"""
Code for task (a) - Crude monte carlo
"""
from parameters import *
np.random.seed(0)
N = 100000
if m == 1:
    sampling_dist = np.random.normal(loc = 0, scale = 1, size = (1,N))
if m == 2:
    sampling_dist = np.random.multivariate_normal(np.zeros(2), sigma_s, size = N
        ).T


def crude_monte_carlo(N, sampling_dist):
```

```
    """
    Crude Monte Carlo simulation to estimate the probability of exceedance of x.

    Parameters:
    N (int): Number of simulations

    Returns:
    float: Estimated probability of exceedance
    """


    # Simulate N outcomes of L
    loss = L(sampling_dist)

    # Mean of indicator converges a.s to P(L > x)
    exeedance = indicator(x, loss)
    # Estimate
    exeedance_prop = np.mean(exeedance)
    # Variance of estimate
    variance = np.var(exeedance) / N
    return exeedance_prop, variance

if __name__ == "__main__":
    print("Crude Monte Carlo simulation: \n")
    # Estimate and variance of estimator
    l, s2 = crude_monte_carlo(N, sampling_dist)
    print("Estimated probability of exceedance of x: ", l)
    print("Variance of MC estimator: ", s2)
```

3 Control Variable (b)

A control variable is now introduced with the aim of reducing the variance of the Monte Carlo estimator. The estimator is now given by

$$\ell_{CV} = Y - \beta(Z - \mathbb{E}[Z]), \tag{1}$$

where $Y = \mathbb{I}(L > x)$ is the original estimator, Z is the control variable and β is a constant. It follows from proposition 5.1, that the optimal value of β is given by

$$\beta^* = \frac{\mathrm{Cov}(Y, Z)}{\mathrm{Var}(Z)}.$$

When considering the choice of a control variable, we should aim to choose a variable that satisfies the following properties:

1. Closed form expression for the expectation of the control variable $\mathbb{E}[Z]$.
2. The control variable should be as correlated with the original estimator Y as possible.
3. The reduction in variance should be significant enough to justify the additional computational cost of calculating the control variable.

Given L, two choices of control variables are natural. The first one is the linear term of L given by:

$$Z_1 = \delta^T \Delta S,$$

and the second one is the quadratic term of L given by:

$$Z_2 = \Delta S^T \Gamma \Delta S.$$

Since m, $\Gamma = 1$, and the fact that ΔS is standard normal, $Z_1 \sim \delta\mathcal{N}(0, 1)$ and $Z_2 \sim \chi^2(1)$, and as such their expectations are given by

$$\mathbb{E}[Z_1] = 0, \quad \mathbb{E}[Z_2] = 1.$$

To test which control variable is the best, a small pilot study is conducted to find the sample correlation between the original estimator Y and the two control variables Z_1 and Z_2, yeilding

$$\text{Corr}(Y, Z_1) \approx 0.25, \quad \text{Corr}(Y, Z_2) \approx 0.37.$$

Given the results of the pilot study, and the fact that Z_2 is not much more computationally expensive than Z_1, Z_2 is chosen as a control variable. This leads to variance reduction facotor of 1.26 compared to the crude MCS. However, the variance reduction scales linearly with the number of samples, and since the reduction is not very significant, and we introduce considerable additional computational cost, the control variable MCS is not a very good choice in this case - we might aswell just double the sample size. The code snippet for the control variable MCS can be seen in Listing 2.

Listing 2: Python code for control variable MCS

```
"""
We introduce a control variable to reduce the varinace.
"""
# Same as in (a)
from a import crude_monte_carlo
from parameters import *
np.random.seed(0)
N = 10000

# Use Q as control variable
def Q(Delta_s,  delta):
    return delta.T @ Delta_s

def compute_optimal_coefficients(Y, Z):
    """
    Y: target, shape (1, N)
    Z: control variates, shape (k, N)
       where k is the number of control variates.

    Returns:
        beta_opt: shape (k,)
    """
    Y = np.asarray(Y).reshape(-1)
    Z = np.atleast_2d(Z)

    if Z.shape[1] != Y.size:
        raise ValueError("Y and Z must have the same number of simulations.")

    cov = np.cov(np.vstack([Y, Z]), ddof=1)

    cov_YZ = cov[0, 1:]
    cov_ZZ = cov[1:, 1:]

    beta_opt = np.linalg.solve(cov_ZZ, cov_YZ)
    return beta_opt

def compute_correlation(Y, Z):
    """
    Compute the sample correlation coefficient for k different control variates.
    Y: target, shape (1, N)
    Z: control variates, shape (k, N)
       where k is the number of control variates.

    Returns:
        correlation: shape (k,)
    """
    Y = np.asarray(Y).reshape(-1)
    Z = np.atleast_2d(Z)

    if Z.shape[1] != Y.size:
        raise ValueError("Y and Z must have the same number of simulations.")
```

```
52
53      cov = np.cov(np.vstack([Y, Z]), ddof=1)
54
55      cov_YZ = cov[0, 1:]
56      var_Y = cov[0, 0]
57      var_Z = np.diag(cov[1:, 1:])
58
59      correlation = cov_YZ / np.sqrt(var_Y * var_Z)
60      return correlation
61
62  def control_monte_carlo(N, sampling_dist):
63      """
64      Control variate Monte Carlo simulation to estimate the probability of
            exceedance of x.
65
66      Parameters:
67      N (int): Number of simulations
68
69      Returns:
70      float: Estimated probability of exceedance
71      """
72
73      # Simulate N outcomes of L
74      loss = L(sampling_dist)
75
76      # Compute control variable
77
78      #Linear term, corr aprrox 0.25
79      # #Z = Q(Delta_s, delta)
80
81      # Quadratic term, corr approx 0.4
82      Z = L(sampling_dist) - Q(sampling_dist, delta)
83      # Expectation of control variable
84      #E_Z = 0 # Linear term is N(0,1) * delta -> E[Z] = 0
85      if m == 1:
86          E_Z = 1 # Quadratic term i Chi_squared(1) -> E[Z] = 1
87      if m == 2:
88          # (f) gives weighting of eigenvalues of two chi_squared(1)
89          E_Z = eigenvalues[0] * 1 + eigenvalues[1] * 1
90
91      # Mean of indicator converges a.s to P(L > x)
92      exeedence = indicator(x, loss)
93
94      # optimal coeffcient
95      beta_opt = compute_optimal_coefficients(exeedence, Z)
96
97      # Control estimator
98      exeedence_tilde = exeedence -  beta_opt*(Z - E_Z)
99
100     # Estimate
101     exeedance_prop = np.mean(exeedence_tilde)
102
103     # Variance of estimate
104     variance = np.var(exeedence_tilde) / N
105
106     return exeedance_prop, variance
107
108 if __name__ == '__main__':
109     # Correlation subrun
110     M = 1000
111     Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size=M).T
112     loss = L(Delta_s)
113     Y = indicator(x, loss)
114     # Compute correlation between linear approximation Q, and second term of L
```

```
    Z = np.zeros((2,M))
    Z[0,:] = Q(Delta_s, delta)
    Z[1,:] = loss - Q(Delta_s, delta)
    correlation = compute_correlation(Y, Z)
    print("Sample Correlation of linear term and exceedance indicator: ",
        correlation[0])
    print("Sample Correlation of quadratic term and exceedance indicator: ",
        correlation[1])

    print("Control variate Monte Carlo simulation: \n")
    Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size=N).T
    # Estimate and variance of estimator
    l_crude, s2_crude = crude_monte_carlo(N, Delta_s)
    l_control, s2_control = control_monte_carlo(N, Delta_s)
    print("Estimated probability of exceedance of x using control variable: ",
        l_control)
    print("Variance of MC estimator using control: ", s2_control)
    print()
    print("Variance reduction factor: ", s2_crude / s2_control)
```

4 Importance-Sampling (c)

Since we are interested in estimating a relatively rare event, importance sampling to reduce the variance is a good candidate for variance reduction. We are interested in designing a new distribution under which the event $L > x$ is likely to occur. For numerical stability, it is desirable to choose a distribution that is not too far from the original distribution. If we inspect L, it is compromised of a linear and a quadratic term. We may consider if we want to shift the mean of ΔS upwards or downwards. Since we have a quadratic term, that is always positive, shifting the mean in a negative direction, will be somewhat dampened by the linear term, as such we would require larger shifting in the negative direction to observe the same effect, therefore, a positive shift is chosen. The expectation of the loss L is given by

$$\mathbb{E}[L] = \mathbb{E}[\delta^T \Delta S + \Delta S^T \Gamma \Delta S] = \mathbb{E}[\delta^T \Delta S] + \mathbb{E}[\Delta S^T \Gamma \Delta S] = 0 + 1 = 1,$$

Now we would like to shift the mean of $\Delta S \sim \mathbb{N}(0,1)$ such that the expectation of L is equal to the threshold $x = 10$, such that the rare event $L > x$, is common. Since ΔS is standard normal, we can shift the mean by adding a constant μ to ΔS, such that $\Delta S + \mu \sim \mathcal{N}(\mu, 1)$. It follows that we can find μ such that the expectation of L under the new distribution $\mathbb{Q}$ to be equal to $x = 10$:

$$\mathbb{E}_{\mathbb{Q}}[L] = \mathbb{E}[\delta^T(\Delta S + \mu) + (\Delta S + \mu)^T \Gamma(\Delta S + \mu)] = 10.$$

as adding a mean μ to ΔS, will shift its mean by μ, so for $m = 1$

$$\mathbb{E}_{\mathbb{Q}}[L] = 2\mu + \mathbb{E}(\Delta S^2) + \mu^2 = 2\mu + 1 + \mu^2 = 10 \implies \mu(\mu + 2) = 9.$$

The positive solution to this equation is $\sqrt{(10)} - 1 \approx 2.16$, and in accordance with above reasoning, we see that a negative shift would require $\mu \approx -4.16$, which is considerably larger in magnitude. Taking $\mu = \sqrt{(10)} - 1$, we get the likelihood ratio

$$\frac{d\mathbb{P}}{d\mathbb{Q}}(\Delta S) = \frac{\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}\Delta S^2}}{\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(\Delta S - \mu)^2}} = e^{-\frac{1}{2}\Delta S^2 + \frac{1}{2}(\Delta S - \mu)^2} = e^{-\mu \Delta S + \frac{1}{2}\mu^2}.$$

Using this design choice, a variance reduction factor of about 30 is achieved. As such, importance-sampling offers a significant improvement compared to crude MCS, far outweighing the additional computational cost of computing the likelihood ratios. The code snippet for the importance sampling MCS can be seen in Listing 3.

Listing 3: Python code for importance sampling MCS

```
"""
```

```
Monte Carlo with importance sampling for variance reduction
"""
from a import indicator, L
from parameters import *
N = 10000
np.random.seed(0)
sampling_dist = np.random.normal(loc = 0, scale = 1, size = (1, N))

def likelihood_ratio(X, mu_p, sigma_p, mu_q, sigma_q):
    """
    We use another normal distribution with greater mean as importance sampling.
    Compute dP(X) / dQ(X), where:
    Under P: X_i ~ N(m_P, variance)
    Under Q: X_i ~ N(m_P + drift, variance)
    """
    # Compute the likelihood ratio
    p = (1 / np.sqrt(2 * np.pi * sigma_p**2)) * np.exp(-0.5 * ((X - mu_p) /
        sigma_p)**2)
    q = (1 / np.sqrt(2 * np.pi * sigma_q**2)) * np.exp(-0.5 * ((X - mu_q) /
        sigma_q)**2)
    return p / q


import numpy as np

def importance_monte_carlo(
    N,
    importance_sample,
    likelihood_ratio_func,
    likelihood_ratio_args=None,
    use_sigma_s = True,
):
    """
    Estimate probability of exceedance using importance sampling.

    Parameters
    ----------
    N : int
        Number of simulations.
    likelihood_ratio_func : callable
        Function that computes likelihood ratios.
    likelihood_ratio_args : dict, optional
        Keyword arguments passed to likelihood_ratio_func.
    importance_sample : ndarray, optional
        Samples drawn from the importance distribution.

    Returns
    estimate of exceedance probability.
    variance of estimator.
    """

    if likelihood_ratio_args is None:
        likelihood_ratio_args = {}

    # Compute likelihood ratios
    weights = likelihood_ratio_func(**likelihood_ratio_args)
    # Diagnosis for LR should be around 1: print("Mean of weights", np.mean(
        weights))
    # Simulate N outcomes of L
    loss = L(importance_sample, use_sigma_s = use_sigma_s)

    # Mean of indicator converges a.s to P(L > x), weighted by the likihood
        ratio
    exeedance = indicator(x, loss) * weights
```

```
    # Estimate
    exeedance_prop = np.mean(exeedance)
    # Variance of estimate
    variance = np.var(exeedance) / N
    return exeedance_prop, variance

if __name__ == '__main__':

    from a import crude_monte_carlo
    l_crude, s2_crude = crude_monte_carlo(N, sampling_dist)
    print("Importance sampling Monte Carlo simulation: \n")
    importance_sample = np.random.multivariate_normal(
        mu_q, sigma_s, size=N
    ).T
    lr_args = {"X" : importance_sample,
               "mu_p" : mu_p,
               "sigma_p" : sigma_p,
               "mu_q" : mu_q,
               "sigma_q" : sigma_q,}

    l_importance, s2_importance = importance_monte_carlo(N,
                                                         importance_sample,
                                                         likelihood_ratio_func =
                                                              likelihood_ratio,
                                                         likelihood_ratio_args=
                                                            lr_args)

    print("Estimated probability of exceedance of x using importance sampling: "
        , l_importance)
    print("Variance of MC estimator using importance sampling: ", s2_importance)
    # Variance reduction factor
    print("The variance reduction factor of importance sampling: ", s2_crude /
        s2_importance)
```

5 Comparison of the methods (d)

To compare the three methods, we examine the confidence intervals for 30 different simulations runs with distinct random seeds, for each of the three methods. The results are shown in Figure 1. The figure shows that the confidence intervals for the crude MCS and control variable MCS differ only slightly, while the confidence intervals for the importance sampling MCS are much narrower. This is consistent with the reduction in variance we observed in the previous sections.

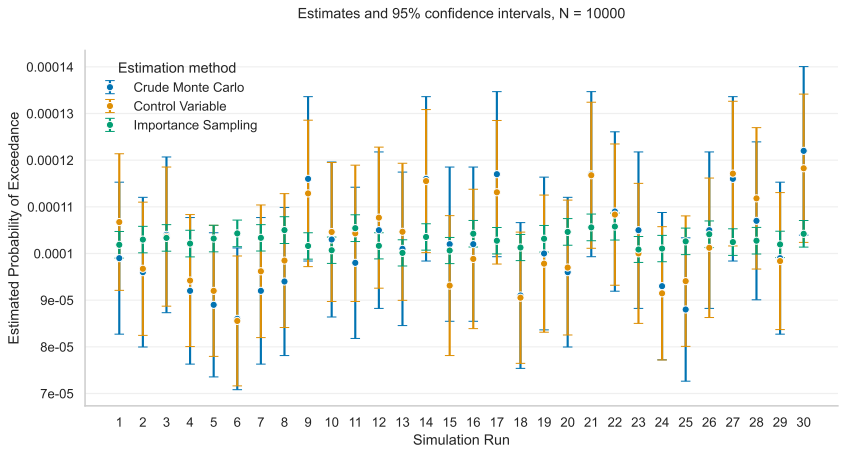


Figure 1: Comparison of confidence intervals for crude MCS, control variable MCS and importance sampling MCS. The confidence intervals are based on 30 simulations with $N = 10^4$ samples each. Dots represent the estimated probabilities.

We see that the confidence intervals are much narrower for the importance sampling MCS, which is consistent with the previous results.
The code to produce the plot can be seen in Listing 4.

Listing 4: Python code for comparison of the three methods

```
import numpy as np
import matplotlib.pyplot as plt
from a import crude_monte_carlo
from b import control_monte_carlo
from c import importance_monte_carlo, likelihood_ratio
from parameters import *
sns.set_theme(style="whitegrid", context="notebook")
from matplotlib.ticker import FuncFormatter

np.random.seed(0)
if m == 2:
    raise ValueError("m must be 1 for (d).")

def confidence_interval(s, l, N, level=0.05):
    """Compute confidence interval, equation (1.2)"""
    z_alpha = norm.ppf(1 - level)
    return (l - z_alpha * s / np.sqrt(N), l + z_alpha * s / np.sqrt(N))

# Column is method, row is simulation run
estimates = np.zeros((SIMULATION_RUNS, len(methods)))
ci_lower = np.zeros_like(estimates)
ci_upper = np.zeros_like(estimates)

# Define seeds for reproducibility
random_seeds = range(0, SIMULATION_RUNS*len(methods))

for j, m in enumerate(methods):
    for i in range(SIMULATION_RUNS):
        np.random.seed(random_seeds[i])
        """
        Simulate the three methods and store the estimates and confidence
            intervals.
        """
        if m == "Crude Monte Carlo":
            # Simulate
            sampling_dist = np.random.normal(loc=0, scale=1, size=(1, Nd))
            l, s2 = crude_monte_carlo(Nd, sampling_dist)
        if m == "Control Variable":

            sampling_dist = np.random.normal(loc=0, scale=1, size=(1, Nd))
            l, s2 = control_monte_carlo(Nd, sampling_dist)
        if m == "Importance Sampling":

            importance_sample = np.random.multivariate_normal(
                mu_q, sigma_s, size=Nd
            ).T
            lr_args = {"X": importance_sample,
                       "mu_p": mu_p,
                       "sigma_p": sigma_p,
                       "mu_q": mu_q,
                       "sigma_q": sigma_q, }
            l, s2 = importance_monte_carlo(Nd,
                                           importance_sample,
                                           likelihood_ratio_func =
                                               likelihood_ratio,
                                           likelihood_ratio_args=lr_args)

        s = np.sqrt(s2)
        # Compute confidence interval
```

```
        ci = confidence_interval(s, l, Nd)
        ci_lower[i, j] = ci[0]
        ci_upper[i, j] = ci[1]
        estimates[i, j] = l

fig, ax = plt.subplots(figsize=(10, 5.5))

# Horizontal offsets to separate the three methods
offsets = [0,0,0]

for j, method in enumerate(methods):
    x = plot_n + offsets[j]

    # Asymmetric error lengths: estimate - lower, upper - estimate
    yerr = np.vstack([
        estimates[:, j] - ci_lower[:, j],
        ci_upper[:, j] - estimates[:, j],
    ])*100

    ax.errorbar(
        x,
        estimates[:, j],
        yerr=yerr,
        fmt="o",
        color=colors[j],
        label=method,
        capsize=4,
        elinewidth=1.8,
        markersize=6,
        markeredgecolor="white",
        markeredgewidth=0.5,
    )

ax.set_xticks(plot_n)
ax.set_xlabel("Simulation Run")
ax.set_ylabel("Estimated Probability of Exceedance")
ax.set_title(f"Estimates and 95% confidence intervals, N = {Nd}\n", pad=14)
ax.legend(title="Estimation method", frameon=False)
ax.grid(axis="y", alpha=0.3)
ax.grid(axis="x", visible=False)

ax.yaxis.set_major_formatter(
    FuncFormatter(lambda value, _: f"{value / 100:g}")
)

sns.despine()
fig.tight_layout()
# Save as SVG
plt.savefig("figures/ConfInts.pdf", format="pdf", bbox_inches="tight")
plt.show()
```

6 Closed form expressison of $\mathbb{P}(L > x)$ for the given parameters (e)

We wish to show that the exceedance probability can be computed in a closed form expression involving the CDF $\phi(x)$ of the standard normal distribution. Given the parameters, $m = 1$, $\delta = 2$, $\Gamma = 1$, $\Sigma_S = 1$ and $x = 10$, we have $\Delta_S \sim \mathcal{N}(0, 1)$ and $L = 2\Delta S + \Delta S^2$. We can rewrite the exceedance probability as

$$\mathbb{P}(L > x) = \mathbb{P}(\Delta S^2 + 2\Delta S - 10 > 0) \tag{2}$$

The left hand side is a quadratic polynomial in ΔS, we note that it is convex, as such, this probability is equivalent to the probablity that ΔS is lesser than the smaller root of the polynomial or greater than

the larger root of the polynomial. The roots of the polynomial are given by

$$r_- = -1 - \sqrt{11}, \quad r_+ = -1 + \sqrt{11}.$$

We note that these events are disjoint, so we can write the exceedance probability as

$$\mathbb{P}(L > x) = \mathbb{P}(\Delta S < r_- \cup \Delta S > r_+) = \mathbb{P}(\Delta S < r_-) + \mathbb{P}(\Delta S > r_+) \tag{3}$$

Since $\Delta S \sim \mathcal{N}(0, 1)$, we can express the exceedance probability in terms of the CDF $\phi(x)$ of the standard normal distribution as

$$\mathbb{P}(L > x) = \phi(r_-) + 1 - \phi(r_+) \approx 0.0103, \tag{4}$$

which is consistent with the results obtained from the Monte Carlo simulations.

6.1 Is this true for other distributions?

There is no guarantee that we can express the exceedance probability in terms of the CDF of the standard normal distribution for other distributions. However, we did not use any specific properties of the standard normal distribution, so if ΔS had a different continous distribtuion, we could still express the exceedance probability in terms of the CDF of that distribution, as long as we can find the roots of the polynomial. That is, if under the new distribution, ΔS has a CDF $F(x)$, we could express the exceedance probability as

$$\mathbb{P}(L > x) = F(r_-) + 1 - F(r_+),$$

6.2 Generalization of crude and control- MCS to other distributions

As long as we can sample from the distribution of $L = h(X)$, where X is a random vector with a known distribution, we can use crude MCS to estimate the exceedance probability without any modifications.

For control variable MCS, we do have the requirement that the expectation of the control variable is known in closed form. As such, the control variable MCS can be generalized to other distributions without any modifications other than that the control variable $Z = g(X)$ has to be chosen such that its expectation $\mathbb{E}[Z]$ is known in closed form.

7 Cholesky Decomposition (f)

We wish to find a matrix $\tilde{C}$ such that $\Delta_S = \tilde{C}Z$, where $Z \sim \mathcal{N}(0, I_2)$, it follows from definition 3.8 that we can find such a matrix $\tilde{C}$ by performing a Cholesky decomposition of the covariance matrix Σ_S. We define the upper triangular matrix $\tilde{C}$ as

$$\tilde{C} = \begin{bmatrix} a & 0 \\ b & c \end{bmatrix},$$

with unknowns $a, b, c \in \mathbb{R}$, and solve

$$\tilde{C}\tilde{C}^T = \begin{bmatrix} a & 0 \\ b & c \end{bmatrix} \begin{bmatrix} a & b \\ 0 & c \end{bmatrix} = \Sigma_S = \begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & 1 \end{bmatrix}.$$

This produces the following system of equations

$$\begin{aligned} a^2 &= 1 \\ ab &= \frac{1}{2} \\ b^2 + c^2 &= 1. \end{aligned}$$

$a^2 = 1 \implies a = 1$, and thus $b = \frac{1}{2}$. We can than solve $\frac{1}{4} - c^2 = 1 \implies c = \sqrt{1 - \frac{1}{4}} = \sqrt{\frac{3}{4}}$.

$$\tilde{C} = \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & \sqrt{\frac{3}{4}} \end{bmatrix}.$$

7.1 Eigen decomposition of $\tilde{C}^T\tilde{C}$

First we compute $\tilde{C}^T\tilde{C}$:

$$\tilde{C}^T\tilde{C} = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & \sqrt{\frac{3}{4}} \end{bmatrix} \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & \sqrt{\frac{3}{4}} \end{bmatrix} = \begin{bmatrix} \frac{5}{4} & \frac{1}{2}\sqrt{\frac{3}{4}} \\ \frac{1}{2}\sqrt{\frac{3}{4}} & \frac{3}{4} \end{bmatrix}.$$

We note that $\tilde{C}^T\tilde{C}$ is a real symmetric matrix, it follows from the spectral theorem that there exists and orthogonal matrix U and a diagonal matrix Λ such that

$$\tilde{C}^T\tilde{C} = U\Lambda U^T. \tag{5}$$

where the columns of U are the eigenvectors of $\tilde{C}^T\tilde{C}$ and the diagonal entries of Λ are the corresponding eigenvalues. Thus, we need to solve the eigenvalue problem

$$\det(\tilde{C}^T\tilde{C} - \lambda I_2) = 0,$$

which yields the characteristic polynomial

$$\left(\frac{5}{4} - \lambda\right)\left(\frac{3}{4} - \lambda\right) - \left(\frac{1}{2}\sqrt{\frac{3}{4}}\right)^2 = 0.$$

This yeilds the eigenvalues $\lambda_1 = \frac{1}{2}$, $\lambda_2 = \frac{3}{2}$, now we find the eigenvectors by solving the system of equations

$$(\tilde{C}^T\tilde{C} - \lambda I_2)u = 0,$$

which yields the eigenvectors

$$u_1 = \begin{bmatrix} \sqrt{\frac{3}{4}} \\ \frac{1}{2} \end{bmatrix}, \quad u_2 = \begin{bmatrix} -\frac{1}{2} \\ \sqrt{\frac{3}{4}} \end{bmatrix}.$$

and U is given as the matrix with columns u_1 and u_2, and Λ is the diagonal matrix with entries λ_1 and λ_2.
The computation is carried out in the code snippet seen in Listing 5.

Listing 5: Python code for eigen decomposition of $\tilde{C}^T\tilde{C}$

```
# Define the matrix
C_T_C = np.array([[0.75, 0.5 * np.sqrt(0.75)],
                  [0.5 * np.sqrt(0.75), 1.25]])

# Compute the eigenvalues and eigenvectors
eigenvalues, U = np.linalg.eig(C_T_C)
eigenvalues = np.real(eigenvalues)
```

7.2 New expression of L

Using our Cholesky decomposition, we have $\Delta S = \tilde{C}Z$, where $Z \sim \mathcal{N}(0, I_2)$. It then follows that we can rewrite L as

$$L = \delta^T\tilde{C}Z + Z^T\tilde{C}^T\Gamma\tilde{C}Z.$$

Now since $\Gamma = I_2$, this simplifies to

$$L = \delta^T\tilde{C}Z + Z^T\tilde{C}^T\tilde{C}Z.$$

We define $Z = [Z_1, Z_2]^T$, and use the eigen decomposition (5) to rewrite L as

$$L = \delta^T\tilde{C}Z + Z^TU\Lambda U^TZ = \delta^T\tilde{C}Z + (U^TZ)^T\Lambda U^TZ. \tag{6}$$

Not let $C = \tilde{C}U \implies C^T = U^T\tilde{C}^T$, and $b = C^T\delta = U^T\tilde{C}^T\delta$. If we look at the first term of L, and use the fact that U is orthogonal, we have

$$\delta^T\tilde{C}Z = (UU^T\tilde{C}^T\delta)^TZ = (Ub)^TZ = b^TU^TZ. \tag{7}$$

Now it is useful to note that the linear transformation $U^T Z \sim \mathcal{N}(0, U^T U)$, orthogonality then gives that $U^T Z \sim \mathcal{N}(0, I_2) = Z$. As such (7) simplifies to

$$\delta^T \tilde{C} Z = b^T U^T Z = b^T Z.$$

Using this property, again we can rewrite (6) as

$$L = b^T Z + (U^T Z)^T \Lambda U^T Z = b^T Z + Z^T \Lambda Z,$$

using (7). The first term is a inner product of two vectors, and the since Λ is diagonal, the second term is a weighted sum of squares of the entries of Z - as such we arrive at the desired expression of L:

$$L = b_1 Z_1 + b_2 Z_2 + \lambda_1 Z_1^2 + \lambda_2 Z_2^2. \tag{8}$$

8 Repetition of (a) and (b) with new parameters

(a) and (b) are repeated with the new parameters, $m = 2$, $\delta = [2, 2]^T$, $\Gamma = I_2$, $\Sigma_S = \begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & 1 \end{bmatrix}$ and $x = 15$. Code wise the implementation is identical, however the m variable in the *parameters.py* file is changed to $m = 2$, this updates δ, Γ and Σ_S accordingly. Thus, the code snippets for (a) and (b) are identical to the previous ones, and can be seen in Listing 1 and Listing 2, only where m is updated to 2 in Listing 8

8.1 Repetition of (a)

For $N = 10^5$, we obtain an estimate of $\hat{\ell} \approx 0.0153$ with a variance of $s^2 \approx 1.5134 \cdot 10^{-7}$.

8.2 Repetition of (b)

For $N = 10^5$, we obtain an estimate of $\hat{\ell} \approx 0.0153$ with a variance of $s^2 \approx 1.1693 \cdot 10^{-7}$. One change in (b) is that we have to update the expectation of the control variable $Z_{control} = \Delta S^T \Gamma \Delta S$. However, it follows from (8) that the expectation of Z is given by

$$\mathbb{E}[Z_{control}] = \mathbb{E}[\lambda_1 Z_1^2 + \lambda_2 Z_2^2] = \lambda_1 \mathbb{E}[Z_1^2] + \lambda_2 \mathbb{E}[Z_2^2] = \lambda_1 \mathbb{E}[\chi_1^2] + \lambda_2 \mathbb{E}[\chi_1^2] = \lambda_1 + \lambda_2 = 2,$$

where $Z_1, Z_2 \sim \mathcal{N}(0, 1)$ and $\lambda_1 = \frac{1}{2}, \lambda_2 = \frac{3}{2}$ are the eigenvalues of $\tilde{C}^T \tilde{C}$, from (f). This is handled automatically if m is set to 2 in the *parameters.py* file. This time we see a variance reduction factor of 1.23, which again is not very significant, and the additional computational cost of computing the control variable is not justified.

9 Exponential tilting (g)

We are now interested in importance sampling similar to (c), but for the new parameters. In the previous section we have shown that we can express L as a function of Z, rather that ΔS, this allows us to perform importance sampling from multivariate normal with identity covariance matrix. Recall that in extension to exercise 27, we discussed that exponential tilting for exponential families, retains distributional form under a new shifted parameter.

$$\int_A \prod_{i \leq 2} f(z) e^{\theta_i^T z - \psi^*(\theta_i)} dz, \tag{9}$$

Our original distribution is $Z \sim \mathcal{N}(0, I_2)$, and we can write the density of Z as

$$f(z) = \frac{1}{2\pi} e^{-\frac{1}{2} z^T(z)}.$$

We can then introduce the titlted distribution with parameter $\theta = (\theta_1, \theta_2) \in \mathbb{R}^2$ as

$$f(z) e^{\theta^T z - \psi^*(\theta)} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2} z^2} e^{\theta_i z - \frac{1}{2}\theta_i^2} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(z - \theta_i)^2}.$$

We see that the new importance sampling distribution from (9) is a joint normal distribution(Q_1, Q_2), where $Q_i \sim \mathcal{N}(\theta_i, 1)$, and $Q_1 \perp Q_2$. As such the likelihood ratio is given by

$$\frac{d\mathbb{P}}{d\mathbb{Q}}(z) = \frac{f(z)}{g(z)} = \frac{\prod_{i\leq 2} f(z_i)}{\prod_{i\leq 2} g(z_i)} = \prod_{i\leq 2} e^{-\theta_i z_i + \frac{1}{2}\theta_i^2}, \tag{10}$$

and we note that we indeed arrive at a new multivariate normal distribution with identity covariance matrix, but with a shifted mean θ. To test different pairs of θ_1, θ_2, i generate a 2D grid of values, and compute the variance and estimate of the importance sampling estimator for the respective pairs. The figures in Figure 2 are generated from the code seen in Listing 9.

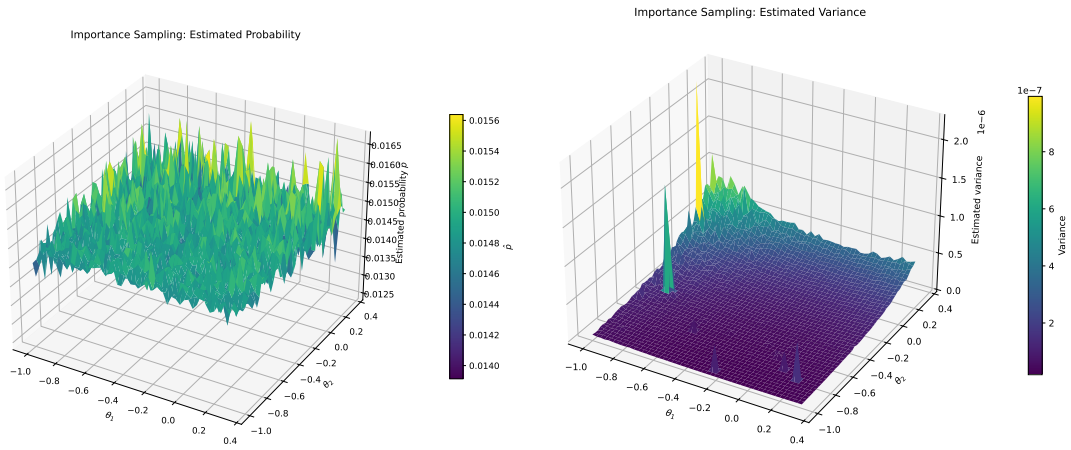


Figure 2: Estimate and variance of the importance sampling estimator for different pairs of θ_1, θ_2., $N = 10^5$ samples.

We see that there is a comparably low varaince region, with estimate values similar to the crude MCS estimate in the region $\theta \in [-0.4, 0.4] \times [-1, -0.4]$. The code for the exponential tilting is seen in Listing 6.

Listing 6: Python code for exponential tilting

```python
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal
from parameters import *
from c import importance_monte_carlo

if m == 1:
    raise ValueError("m must be 2 for (g).")

N = 10000
np.random.seed(0)

def importance_sample(N, theta):
    return np.random.multivariate_normal(
        mean=theta,
        cov=np.eye(m),
        size=N
    ).T


def likelihood_ratio(importance_sample, theta):
    """
    We use another normal distribution with shifted mean as importance sampling.
    Compute dP(X) / dQ(X), where:
    Under P: X_i ~ N(0, I2)
    Under Q: X_i ~ N(theta, I2)
    """
    x = importance_sample
    theta = theta[:, None]  # shape (2, 1)
```

```
    # Follows from (f):
    lr = np.exp(-theta * x + 0.5 * theta**2)
    lr = np.prod(lr, axis=0)
    return lr


np.random.seed(0)
N = 10**5
results = {}
theta_values = np.linspace(-1, 1/3, 50)

# Generate grid of theta pairs
thetas = np.array([
    [t1, t2]
    for t1 in theta_values
    for t2 in theta_values
])

# Compute estimates
for theta in thetas:
    imp_sample = importance_sample(N, theta)
    estimate, s2 = importance_monte_carlo(
        N,
        importance_sample=imp_sample,
        likelihood_ratio_func=likelihood_ratio,
        likelihood_ratio_args={
            "importance_sample": imp_sample,
            "theta": theta
        },
        use_sigma_s=False
    )

    results[tuple(theta)] = (estimate, s2)


# --- 3D surface plot of estimated variance ---
from plots import *
fig, ax = three_d_estimate(theta_values, results)
fig.savefig("figures/2Dest.pdf", bbox_inches="tight")
plt.close(fig)

fig, ax = three_d_variance(theta_values, results)
fig.savefig("figures/2Dvar.pdf", bbox_inches="tight")
plt.close(fig)
```

10 Cumlant generating function of L (h)

We wish to show that the cumulant generating function of L is given by

$$\psi_L(\theta) = \log \mathbb{E}[e^{\theta L}] = \frac{1}{2} \sum_{i=1}^{2} \left(\frac{\theta^2 \beta_i^2}{1 - 2\theta\lambda_i} - \log(1 - 2\theta\lambda_i) \right).$$

We recall that $b = C^T \delta$, and λ_i are the eigenvalues of $\tilde{C}^T \tilde{C}$. To show the relation above, we introduce the following constant $\gamma_i = \dfrac{b_i}{2\lambda_i}$. Without proof, we are given that

$$\log \mathbb{E}[e^{\theta(Z_i + c)^2}] = (1 - 2\theta)^{-1/2} e^{\frac{\theta c^2}{1 - 2\theta}}, \tag{11}$$

which holds for $c \in \mathbb{R}$ and $\theta < \frac{1}{2}$. Now, the idea is to rewrite L as a sum of two terms if the form $(Z_i + c)^2$, and then use the relation above to compute the cumulant generating function of L. Let

$$M_i = (Z_i + \frac{b_i}{2\lambda_i})^2 = Z_i^2 + \frac{b_i^2}{4\lambda_i^2} + \frac{b_i}{\lambda_i} Z_i.$$

If we then define L_i as M_i subtracted by γ_i, then

$$L_i = M_i - \gamma_i^2,$$

where one then finds that that $L = \sum_{i=1}^{2} \lambda_i L_i$ - Note that $L_1 \perp L_2$. We can then compute the cumulant generating function of L as. Now let $\theta_i = \theta\lambda_i$, then

$$\psi_L(\theta) = \log \mathbb{E}[e^{\theta L}] = \log \mathbb{E}[e^{\sum_{i=1}^{2} \theta_i L_i}] = \log \mathbb{E}[e^{\theta_1 L_1} e^{\theta_2 L_2}]$$

since $L_1 \perp L_2$, we can write

$$\psi_L(\theta) = \log \mathbb{E}[e^{\theta_1 L_1}]\mathbb{E}[e^{\theta_2 L_2}] = \log \mathbb{E}[e^{\theta_1 L_1}] + \log \mathbb{E}[e^{\theta_2 L_2}] =$$

$$\log \mathbb{E}[e^{\theta_1 M_1}] + \log \mathbb{E}[e^{\theta_2 M_2}] - \log \mathbb{E}[e^{\theta_1 \gamma_1}] - \log \mathbb{E}[e^{\theta_2 \gamma_2}] =$$

$$\log \mathbb{E}[e^{\theta_1 M_1}] + \log \mathbb{E}[e^{\theta_2 M_2}] - \theta_1 \gamma_1^2 - \theta_2 \gamma_2^2.$$

Now we assume $\theta < \frac{1}{2}$ and use (11) to compute the terms containing M_i, and we find that above equation simplifies to

$$\psi_L(\theta) = \sum_{i=1}^{2} \log(\mathbb{E}[e^{\theta_i M_i}]) - \theta_i \gamma_i = \sum_{i=1}^{2} \left(-\frac{1}{2} \log(1 - 2\theta_i) + \frac{\theta_i \gamma_i^2}{(1 - 2\theta_i)} - \theta_i \gamma_i^2 \right) =$$

$$\sum_{i=1}^{2} \left(-\frac{1}{2} \log(1 - 2\theta_i) + \frac{\theta_i \gamma_i^2 - \theta_i \gamma_i^2 (1 - 2\theta_i)}{(1 - 2\theta_i)} \right) = \sum_{i=1}^{2} \left(-\frac{1}{2} \log(1 - 2\theta_i) + \frac{2\theta_i^2 \gamma_i^2}{(1 - 2\theta_i)} \right),$$

finally, we can substitute back $\theta_i = \theta\lambda_i$ and $\gamma_i = \frac{b_i}{2\lambda_i}$ to obtain the desired result

$$\sum_{i=1}^{2} \left(-\frac{1}{2} \log(1 - 2\theta_i) + \frac{2\theta^2 \lambda_i^2 \frac{b_i^2}{4\lambda_i^2}}{(1 - 2\theta\lambda_i)} \right) = \frac{1}{2} \sum_{i=1}^{2} \left(\frac{\theta^2 b_i^2}{(1 - 2\theta\lambda_i)} - \log(1 - 2\theta\lambda_i) \right) \square$$

11 Choosing θ based on the cumulant generating function (i)

One heuristic for choosing θ, is to choose it in such a manner that the event $L > x$ is no longer rare. From section 6 in the SupplNotes, we know that the mean under the tilted distibution can be found as

$$\psi_L'(\theta) = \mathbb{E}_\theta[L],$$

so we can solve for θ such that $\psi_L'(\theta) = x = 15$, such that this event is the expected value under the new distribution. We find

$$\psi_L'(\theta) = \frac{1}{2} \sum_{i=1}^{2} \left(\frac{2\theta b_i^2 (1 - 2\theta\lambda_i) + 2\theta^2 b_i^2 \lambda_i}{(1 - 2\theta\lambda_i)^2} + \frac{2\lambda_i}{(1 - 2\theta\lambda_i)} \right) = \sum_{i=1}^{2} \frac{\theta b_i^2 (1 - \theta\lambda_i)}{(1 - 2\theta\lambda_i)^2} + \frac{\lambda_i}{(1 - 2\theta\lambda_i)}.$$

We can now solve for θ such that $\psi_L'(\theta) = 15$, under the constraint that $max(\lambda_i)\theta < \frac{1}{2} \implies \theta < \frac{1}{3}$, since $max(\lambda_i) = \frac{3}{2}$.

11.1 Distribution under the shifted measure

Before we set out to solve for θ, we can also compute the distribution of L under the new measure. We have that

$$\mathbb{P}_\theta(B) = \int_B f(z) e^{\theta L - \psi_L(\theta)} dz,$$

since $\psi_L(\theta)$ is independent of z, we can write

$$\mathbb{P}_\theta(B) = \int_B f(z) e^{\theta L} dz e^{-\psi_L(\theta)} = e^{-\psi_L(\theta)} \int_B f(z) e^{\theta L} dz,$$

where $e^{-\psi_L(\theta)}$ is a normalization constant. If we look at the integrand, in the latter integral, and recall that $L(z) = b^T z + z^T \Lambda z$, we can write

$$f(z)e^{\theta L} = \frac{1}{2\pi} e^{-\frac{1}{2}z^T z} e^{\theta(b^T z + z^T \Lambda z)} = \frac{1}{2\pi} e^{-\frac{1}{2}z^T z + \theta b^T z + \theta z^T \Lambda z} = e^{-\frac{1}{2}z^T (I_2 - 2\theta\Lambda) z + \theta b^T z},$$

We wish to complete the square, let $A = I_2 - 2\theta\Lambda$, we can write the exponent as

$$-\frac{1}{2}z^T A z + \theta b^T z = -\frac{1}{2}(z - A^{-1}\theta b)^T A (z - A^{-1}\theta b) + \frac{1}{2}\theta b^T A^{-1} \theta b.$$

thus we get the normal kernel

$$e^{-\frac{1}{2}(z - A^{-1}\theta b)^T A (z - A^{-1}\theta b)}$$

and thus the distribution of L under the new measure is given by

$$e^{\psi_L(\theta)} f(z) e^{\theta L} \sim N(\theta A^{-1} b, A^{-1}).$$

11.2 Algorithm

We can now use above results to implement an algorithm for importance sampling is as follows:

1. Solve for θ such that $\psi_L'(\theta) = x$, under the constraint that $max(\lambda_i)\theta < \frac{1}{2}$.
2. Compute the distribution of L under the new measure, which is given by $N(\theta A^{-1} b, A^{-1})$.
3. Sample from the new distribution and compute the likelihood ratio.
4. Compute the estimate of the exceedance probability using the samples and likelihood ratios.

Note that in step 1, we must ensure that the solution for θ that we find using a root-finding method, obeys the constraint that $max(\lambda_i)\theta < \frac{1}{2}$. For step 3, we recall that the likelihood ratio is given by the inverse of the exponential tilting, which is given by

$$\frac{d\mathbb{P}}{d\mathbb{Q}}(z) = e^{-\theta L + \psi_L(\theta)}.$$

this can be computed once we have determined θ and drawn samples from the importance sampling distribution. The code for the algorithm is seen in Listing 7. The numerical solver finds $\theta^* = 0.20315$, which is within the constraint $\theta < \frac{1}{3}$.
For $N = 10^5$, we obtain an estimate of $\hat{\ell} \approx 0.0143$ with a variance of $s^2 \approx 1.015 \cdot 10^{-7}$, which is consistent with the results obtained from the crude MCS. If we compare the variance of the crude MCS we get a varince reduction factor of

$$\frac{1.5134 \cdot 10^{-7}}{1.015 \cdot 10^{-7}} \approx 1.5,$$

we note that this is much lower compared to the variance reduction when $m = 1$.

Listing 7: Python code for importance sampling using exponential tilting

```
import numpy as np
from scipy.optimize import fsolve
from c import importance_monte_carlo
from parameters import *
if m != 2:
    raise ValueError("m must be 2 for (i).")
N = 10**4
np.random.seed(0)

# Step 1. Solve for theta
def psi_prime_L(theta):
    """Solve for theta in the first derivative of the cumulant generating
       function of L
    such that the shifted distribution will have mean x"""
    lam = eigenvalues
    return (np.sum(theta*b**2*(1-theta*lam) /
            (1-2*theta*lam)**2 +
```

```
            lam /
            (1-2*theta*lam))) - 15

theta_star = fsolve(psi_prime_L, 0.1)[0]


# Step 2. Compute sampling distribution
def sampling_distribution(theta, N):
    """Sampling distribution for importance sampling"""

    A_inv = np.diag(1 / (1 - 2 * theta * eigenvalues))

    mu = (theta*A_inv @ b).flatten()

    return np.random.multivariate_normal(mean=mu, cov=A_inv, size=N).T


# Step 3. Define functions to compute maximum likelihood ratio
def psi_L(theta):
    """Cumulant Generating Function of L - use to compute likelihood ratio for
        importance sampling"""
    lam = eigenvalues
    return 0.5 * np.sum(theta ** 2 * b ** 2 /
                        (1 - 2 * theta * lam) -
                        np.log(1 - 2 * theta * lam)
                        )

def likelihood_ratio(theta, X):
    """Likelihood ratio for importance sampling"""

    return np.exp(-theta*L(X, use_sigma_s= False) + psi_L(theta))

# Step 4. compute estimates
if __name__ == '__main__':
    print("Theta_star for importance sampling: ", theta_star)
    print("Theta condition holds: ", theta_star*np.maximum(eigenvalues[0],
        eigenvalues[1]) < 1/2)
    print("Importance sampling Monte Carlo simulation for (h): \n")
    importance_sample = sampling_distribution(theta_star, N)
    lr_args = {"X" : importance_sample,
               "theta" : theta_star
               }

    l_importance, s2_importance = importance_monte_carlo(N,
                                                         importance_sample,
                                                         likelihood_ratio_func =
                                                              likelihood_ratio,
                                                         likelihood_ratio_args=
                                                             lr_args,
                                                         use_sigma_s= False)

    print("Estimated probability of exceedance of x using importance sampling: "
        , l_importance)
    print("Variance of MC estimator using importance sampling: ", s2_importance)
```

12 Stratified sampling (j)

The issue we are trying to address is that the event $L > x$ is rare, and as such, we need large samples in order to obtain a good estimate, since we may face a scenario where no samples exceed the threshold x. To adress this we could also use stratified sampling. Instead of sampling from the orignal distribution of L, we can sample from conditional distributions of L given that it falls within certain strata. The drawback of the method is that we need to know the conditional distributions of L given $Q = i$, and the

probabilities $\mathbb{P}(Q = i)$.

Suppose that we can tackle these obstacles: One choice of a stratification variable Q is to define it as

$$Q = \begin{cases} 1, & \text{if } Z_1 < a \\ 2, & \text{if } a \leq Z_1 < b \\ 3, & \text{if } Z_1 \geq b \end{cases}$$

such that for suitable choices of a and b, we have that $\mathbb{P}(L > x|Q = i)$ is no longer rare for atleast one value of i. For this choice, we at the very least know the probabilities $\mathbb{P}(Q = i)$. We can then use the law of total probability to write

$$\mathbb{P}(L > x) = \sum_{i=1}^{3} \mathbb{P}(L > x|Q = i)\mathbb{P}(Q = i).$$

which we estimate by the indicator function

$$\hat{\ell}^{(s)} = \sum_{i=1}^{3} \frac{\mathbb{P}(Q = i)}{N_i} \sum_{j=1}^{N_i} \mathbb{I}(L_{ij} > x)$$

Where L_{ij} is the j-th sample from the conditional distribution of L given that $Q = i$, and N_i is the number of samples drawn from aforementioned conditional distribution. AS such, the estimate is obtained by averaging the indicator functions over the samples weighted by the probabilities of belonging to each stratum. How many samples are allocated to each strata is then a matter of either using approximate optimal allocation, or using proportional allocation.

13 Appendix

Listing 8: parameters.py - Parameters for the various code snippets

```python
import numpy as np

### (a), (b) ###
# Set m = 1 for (a) and (b) and m = 2 for repetition of (a) and (b) with m = 2
m = 2

# delta, gamma is a column vector of size m
delta = np.ones((m, 1)) * 2
gamma = np.eye(m)

def indicator(x, L):
    # Exceedance of x
    return L > x

if m == 1:

    sigma_s = np.ones((m, m))

    x = 10

    def L(X, use_sigma_s = True):
        """L for N simulations, keyword added for interchangeability of m = 1 or
            2
        input:
        N (int): Number of simulations
        X: of shape (m, N) samples
        gamma: vector of size m
        delta: vector of size m
        """
        # Returns (N,) losses

```

```
        L = delta.T @ X + np.sum(X * (gamma @ X), axis=0)
        return L

    ### (c) ###

    # Q measure paramters
    mu_q = np.ones(m) * np.sqrt(10) - 1
    sigma_q = sigma_s

    # P measure parameters
    mu_p = np.zeros(m)
    sigma_p = sigma_s

    ######## (d) ########
    import seaborn as sns
    from scipy.stats import norm

    Nd = 10000
    SIMULATION_RUNS = 30
    plot_n = np.arange(1, SIMULATION_RUNS + 1)
    methods = ["Crude Monte Carlo", "Control Variable", "Importance Sampling"]
    colors = sns.color_palette("colorblind", 3)


if m == 2:
    # Updates the parameters accordingly, when we run the simulations

    sigma_s = np.array([[1, 0.5], [0.5, 1]])
    x = 15

    # For use in (g) and (h)
    # Define the matrix
    C_T_C = np.array([[0.75, 0.5 * np.sqrt(0.75)],
                      [0.5 * np.sqrt(0.75), 1.25]])

    # Compute the eigenvalues and eigenvectors
    eigenvalues, U = np.linalg.eig(C_T_C)
    eigenvalues = np.real(eigenvalues)

    C_tilde = np.array([[1, 1 / 2],
                        [0, np.sqrt(3 / 4)]])
    C = C_tilde @ U
    b = np.real(C.T @ delta).flatten()

    def L(X, use_sigma_s = True):
        """L for N simulations
        input:
        N (int): Number of simulations
        X: of shape (m, N) samples
        gamma: vector of size m
        delta: vector of size m
        """
        if use_sigma_s:
            L = delta.T @ X + np.sum(X * (gamma @ X), axis=0)
            return L
        else:
            # Loss computed in accordance with (f)
            lam1 = np.real(eigenvalues[0])
            lam2 = np.real(eigenvalues[1])
            L = b[0] * X[0, :] + b[1] * X[1, :] + lam1 * np.pow(X[0, :], 2) +
                lam2 * np.pow(X[1 , :], 2)
            return L
```

Listing 9: Plotting code

```python
"""3D plotting helpers for importance-sampling results.
DISCLAIMER: VISUALIZATION CODE MADE BY AI.
Expected results format:
    results[(theta1, theta2)] = (estimate, variance)

Example from g.py:
    from plots import three_d_estimate, three_d_variance

    three_d_estimate(theta_values, results)
    three_d_variance(theta_values, results)

The aliases `3destimate` and `3dvariance` are not valid Python identifiers
because an identifier cannot start with a digit. Use `three_d_estimate`
and `three_d_variance` instead.
"""

import numpy as np
import matplotlib.pyplot as plt


def _make_grid(theta_values, results, value_index):
    """Build a grid from results, preserving the order in theta_values."""
    theta_values = list(theta_values)
    n = len(theta_values)

    grid = np.empty((n, n), dtype=float)
    for i, theta1 in enumerate(theta_values):
        for j, theta2 in enumerate(theta_values):
            grid[i, j] = results[(theta1, theta2)][value_index]

    return theta_values, grid


def three_d_estimate(theta_values, results, *, ax=None, show=True):
    """Plot the estimated probability (tuple element 0) over the theta grid."""
    theta_values, estimate_grid = _make_grid(theta_values, results, 0)
    theta1_grid, theta2_grid = np.meshgrid(theta_values, theta_values, indexing="ij")

    created_fig = ax is None
    if created_fig:
        fig = plt.figure(figsize=(10, 7))
        ax = fig.add_subplot(111, projection="3d")
    else:
        fig = ax.figure

    surface = ax.plot_surface(
        theta1_grid, theta2_grid, estimate_grid,
        cmap="viridis", edgecolor="none"
    )
    ax.set_xlabel(r"$\theta_1$")
    ax.set_ylabel(r"$\theta_2$")
    ax.set_zlabel(r"Estimated probability $\hat{p}$")
    ax.set_title("Importance Sampling: Estimated Probability")
    fig.colorbar(surface, ax=ax, shrink=0.65, pad=0.1, label=r"$\hat{p}$")
    fig.tight_layout()

    if show:
        plt.show()
    return fig, ax


def three_d_variance(theta_values, results, *, ax=None, show=True):
```

```
    """Plot the estimated variance (tuple element 1) over the theta grid."""
    theta_values, variance_grid = _make_grid(theta_values, results, 1)
    theta1_grid, theta2_grid = np.meshgrid(theta_values, theta_values, indexing=
        "ij")

    created_fig = ax is None
    if created_fig:
        fig = plt.figure(figsize=(10, 7))
        ax = fig.add_subplot(111, projection="3d")
    else:
        fig = ax.figure

    surface = ax.plot_surface(
        theta1_grid, theta2_grid, variance_grid,
        cmap="viridis", edgecolor="none"
    )
    ax.set_xlabel(r"$\theta_1$")
    ax.set_ylabel(r"$\theta_2$")
    ax.set_zlabel("Estimated variance")
    ax.set_title("Importance Sampling: Estimated Variance")
    fig.colorbar(surface, ax=ax, shrink=0.65, pad=0.1, label="Variance")
    fig.tight_layout()

    if show:
        plt.show()
    return fig, ax


# Short aliases that are valid Python identifiers.
plot_3d_estimate = three_d_estimate
plot_3d_variance = three_d_variance
```